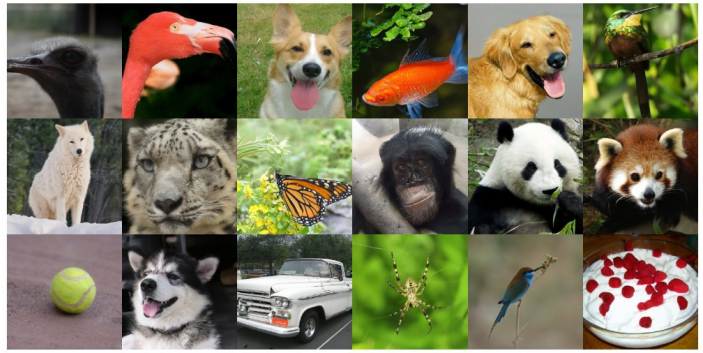


Diffusion Models

From noise to images: how modern generators work

cnrs IDRIS

Why diffusion models?



Class-conditional generation

Dhariwal & Nichol, 2021



Text → image

Stable Diffusion (Rombach et al., 2022)



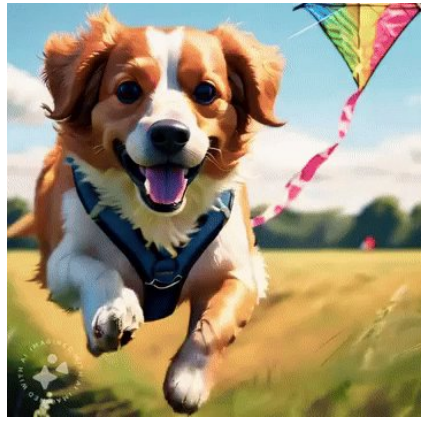
Outpainting

DALL·E 2 — August Kamp × OpenAI, 2022

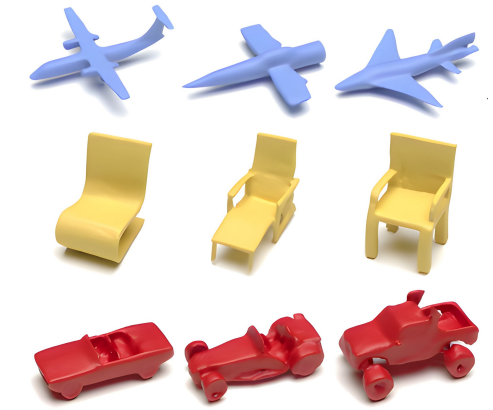
- Major family of image and video generators
- Generate: from a class label, from a text prompt, from an existing image
- The model synthesizes an output from what it learned

Same recipe behind all three: from pure noise to a new image

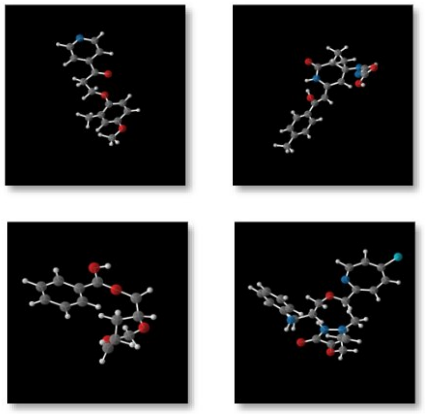
Beyond images: video, 3D, molecules, science



Video — Emu Video (Meta)



3D shapes — LION



Molecules — GeoLDM

...also in Science:

AlphaFold 3: structure

RFdiffusion: proteins

GenCast: weather

- Same recipe, different data:
  - video = images + time
  - shape = 3D coordinates
  - molecule = atom types + 3D coordinates
- Representations and noising rules depend on the data
- Goal: samples that are **realistic AND diverse,** not one averaged answer

Sources: ai.meta.com/blog/emu-text-to-video-generation-image-editing-research · arxiv.org/pdf/2210.06978 (LION) · arxiv.org/pdf/2305.01140 (GeoLDM)

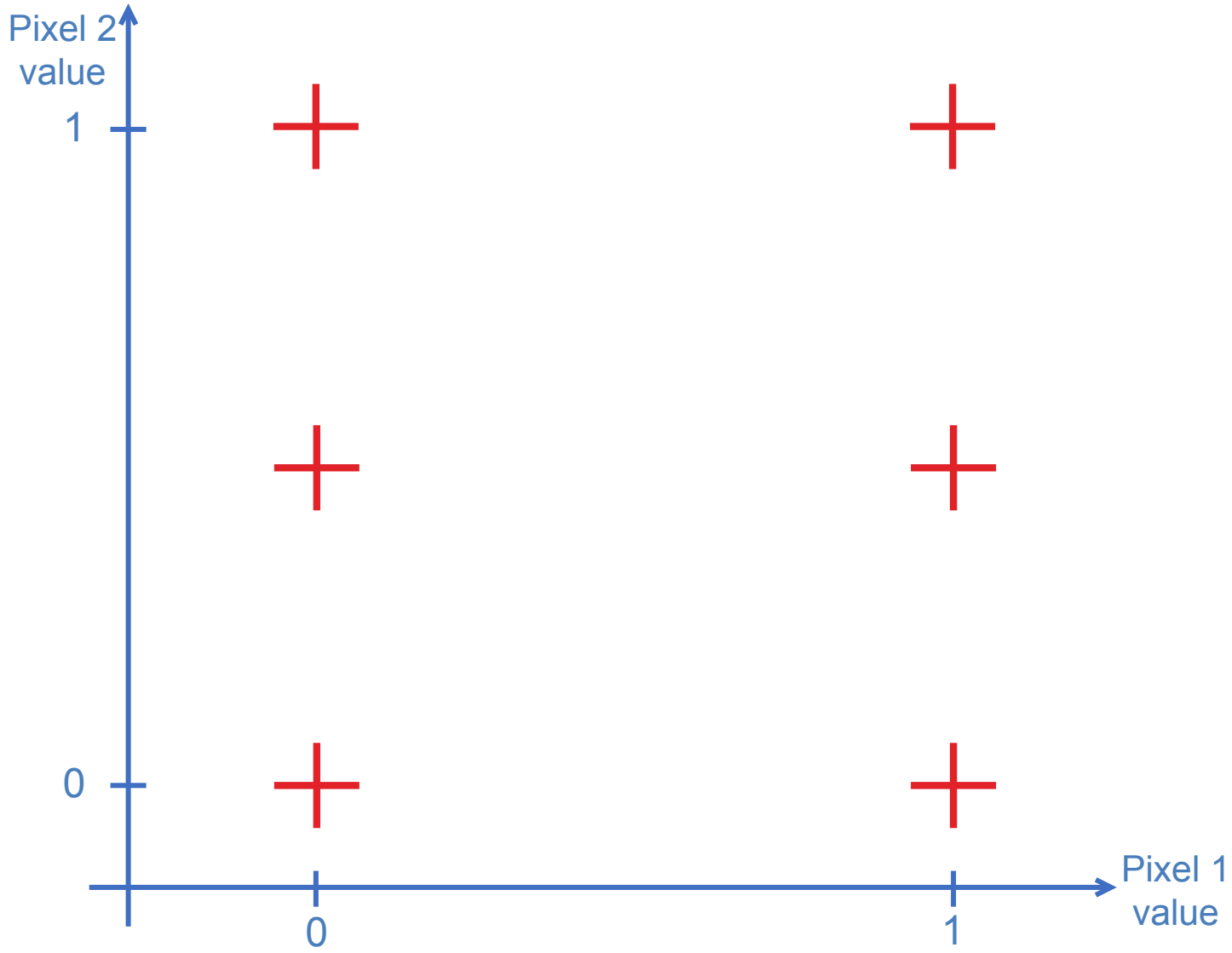
What is an image ?



$$\underbrace{N_{width} \times N_{height}}_{N_{pixels}} \times N_{channels} = \text{total number of values that defines an image (big number)}$$

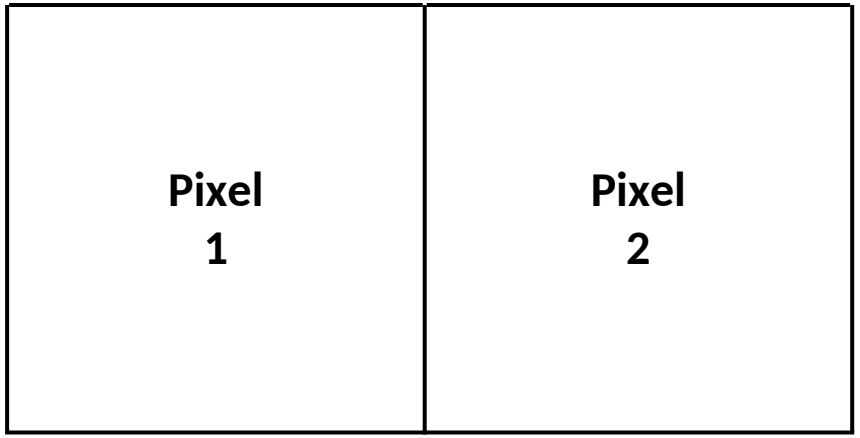
→ **Point (or vector) in a (very) high dimensional space**

What is an image ? Simple case

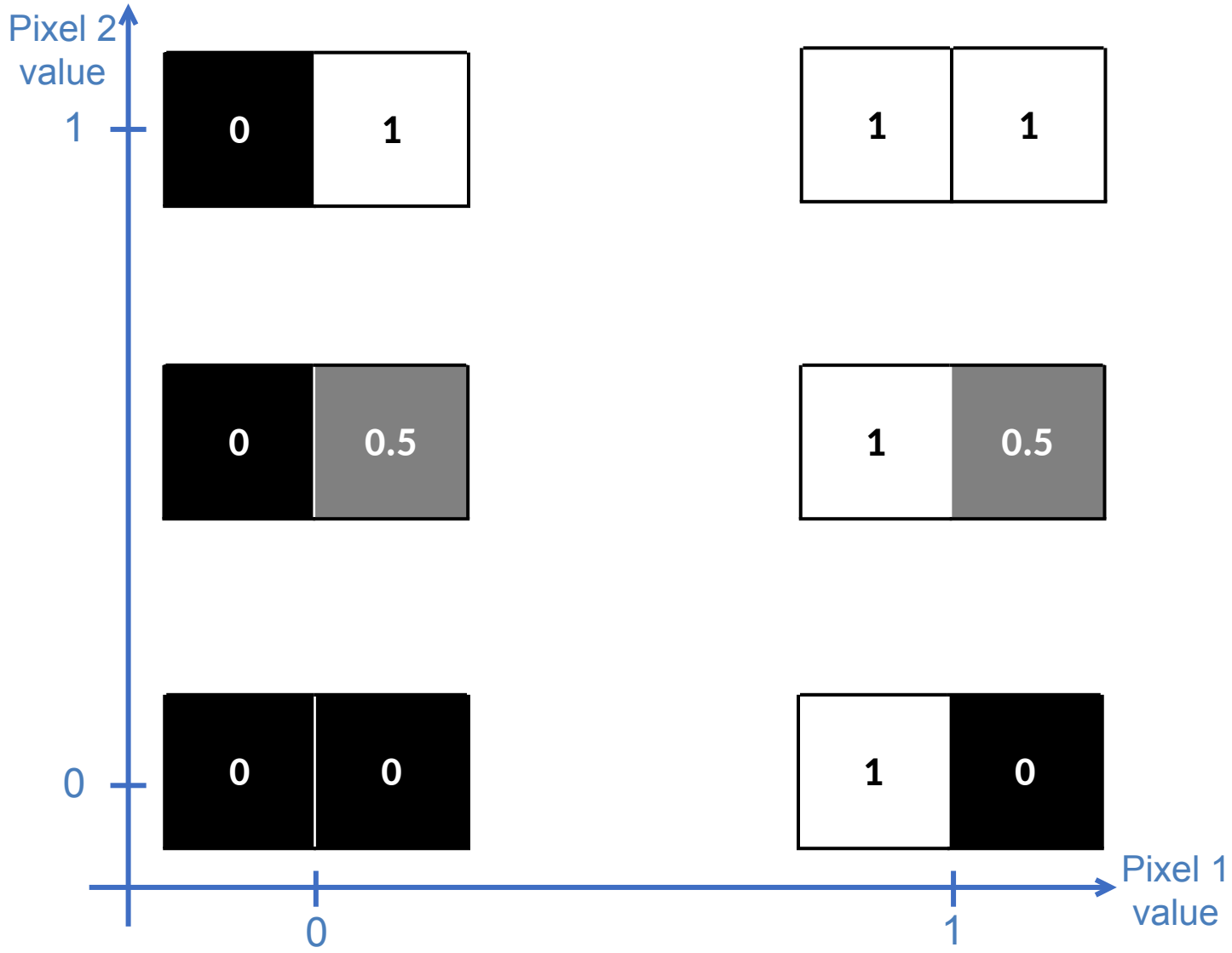


$N_{width} = 2$, $N_{height} = 1$, $N_{channels} = 1$

Each image → a point in 2D

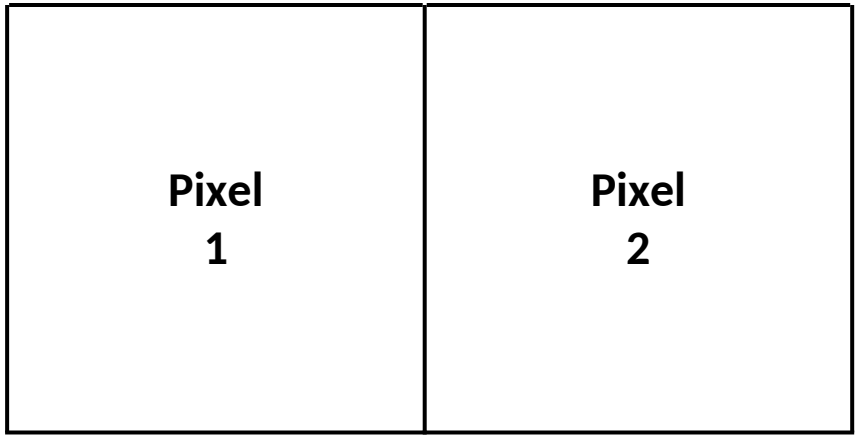


What is an image ? Simple case



$N_{width} = 2$, $N_{height} = 1$, $N_{channels} = 1$

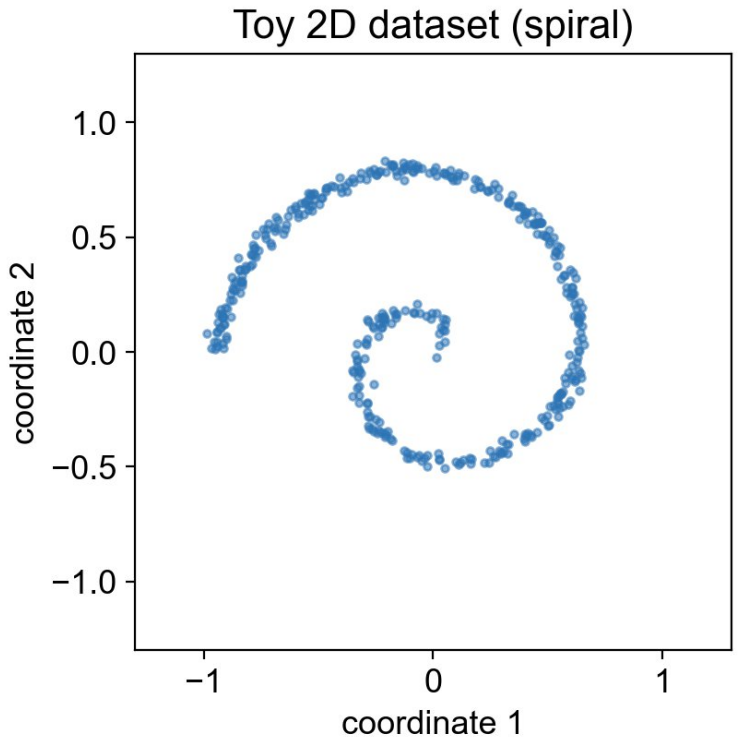
Each image → a point in 2D



A dataset is a cloud of points

One image = one point/vector





All pictures of the Jean Zay cluster “live” in a particular structure, a “cloud” in a very high-dimensional space

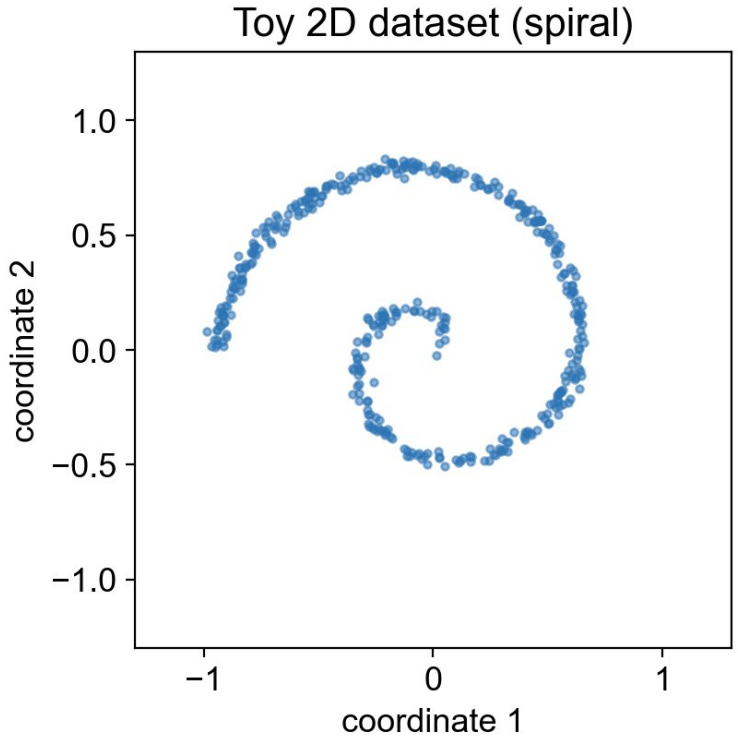
A dataset is a cloud of points

One image = one point/vector = x

q(**x**): density of real data
(q(**x**)~1 means **x** resemble the object of interest)

Goal: generate samples with the same statistical structure

But: we observe examples, not the distribution



Toy 2D distribution — one point per sample

Important note: random point (almost) always look like noise

Classification vs generation

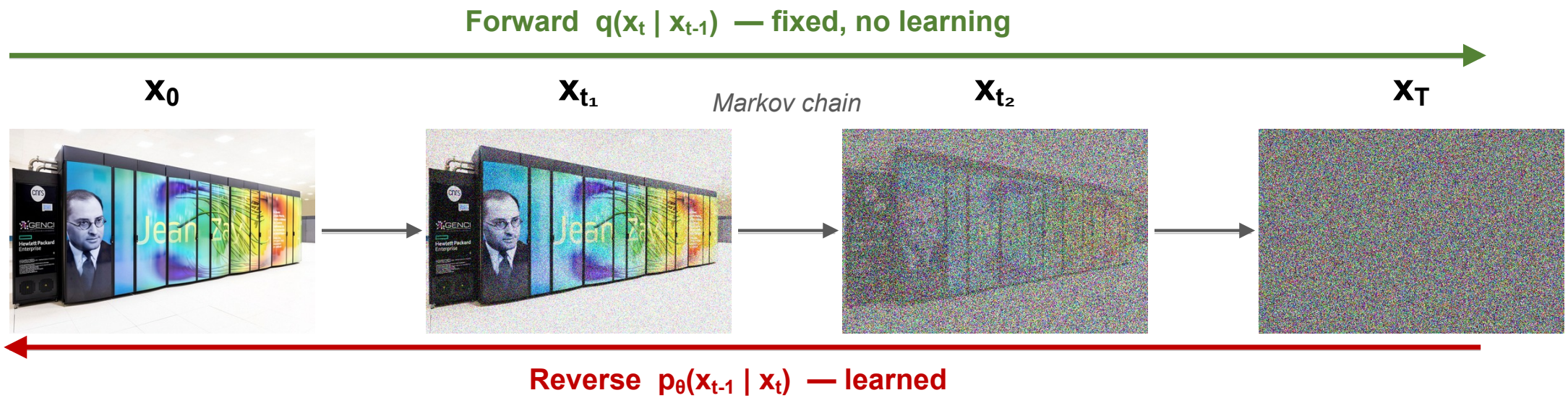
Classification	(Unconditional) generation
Training data: pairs (x, y)	**Training data:** images **x**, without class labels
Model: $p_\varphi(y \mid x) \approx q(y \mid x)$	**Model:** $p_\theta(x) \approx q(x)$
Loss: cross-entropy vs a one-hot target	**Loss:** ? what would the target even be?
Output: class distribution → argmax	**Output:** a **new sample**, not a label

Note: p_θ = learned model, θ = learned parameters

Predicting an average image is not the same as sampling:
we want to model the range of possible answers, not just one average
➡ that's where diffusion excels

Diffusion = two processes

Here: focus on Denoising Diffusion Probabilistic Models (DDPM, Ho et al. 2020)



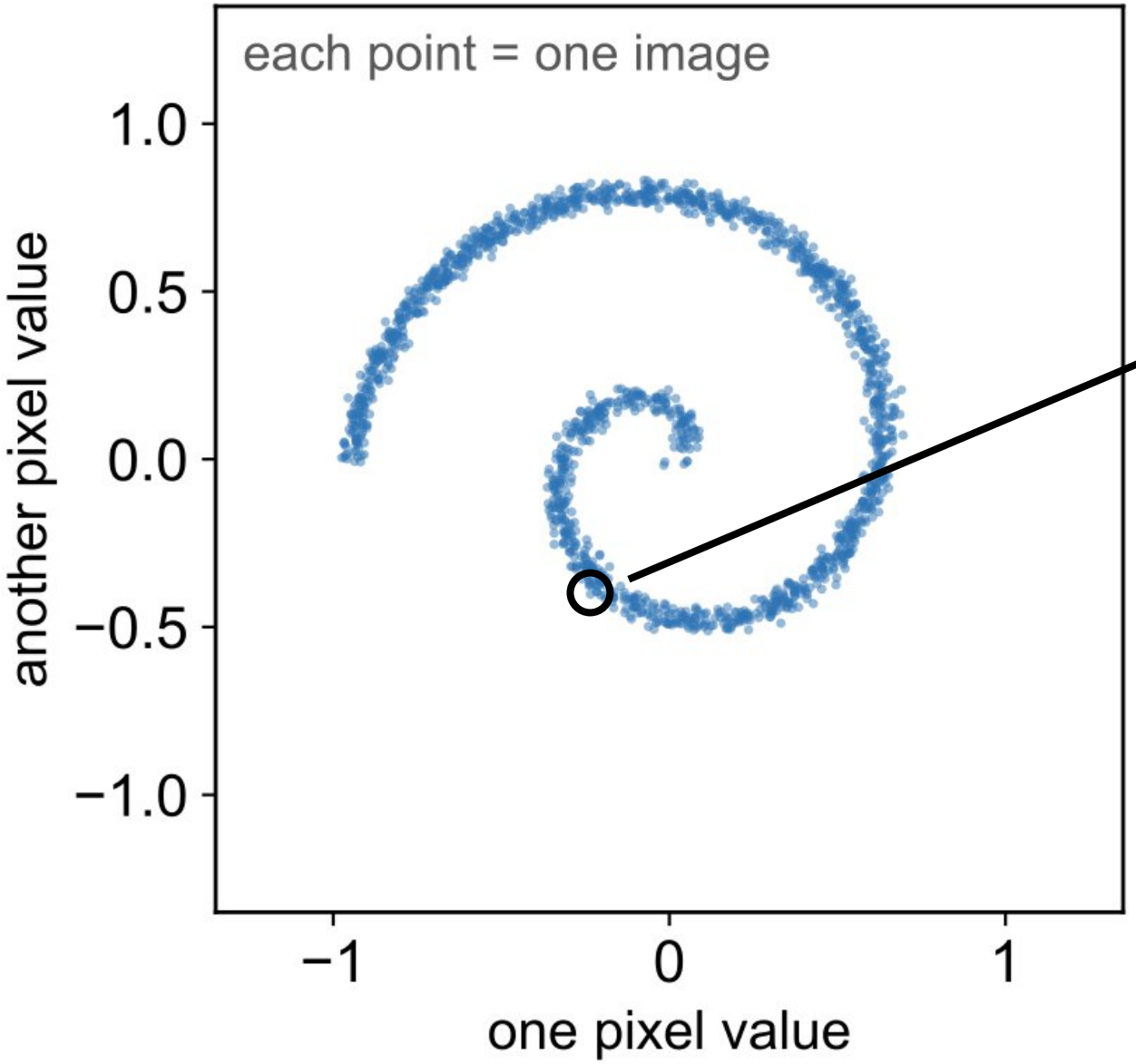
Forward (fixed): add a little noise, T times, nothing to learn

Reverse model: learn transitions back toward plausible data (not the exact steps)

Markov chain: each step only depends on the previous one

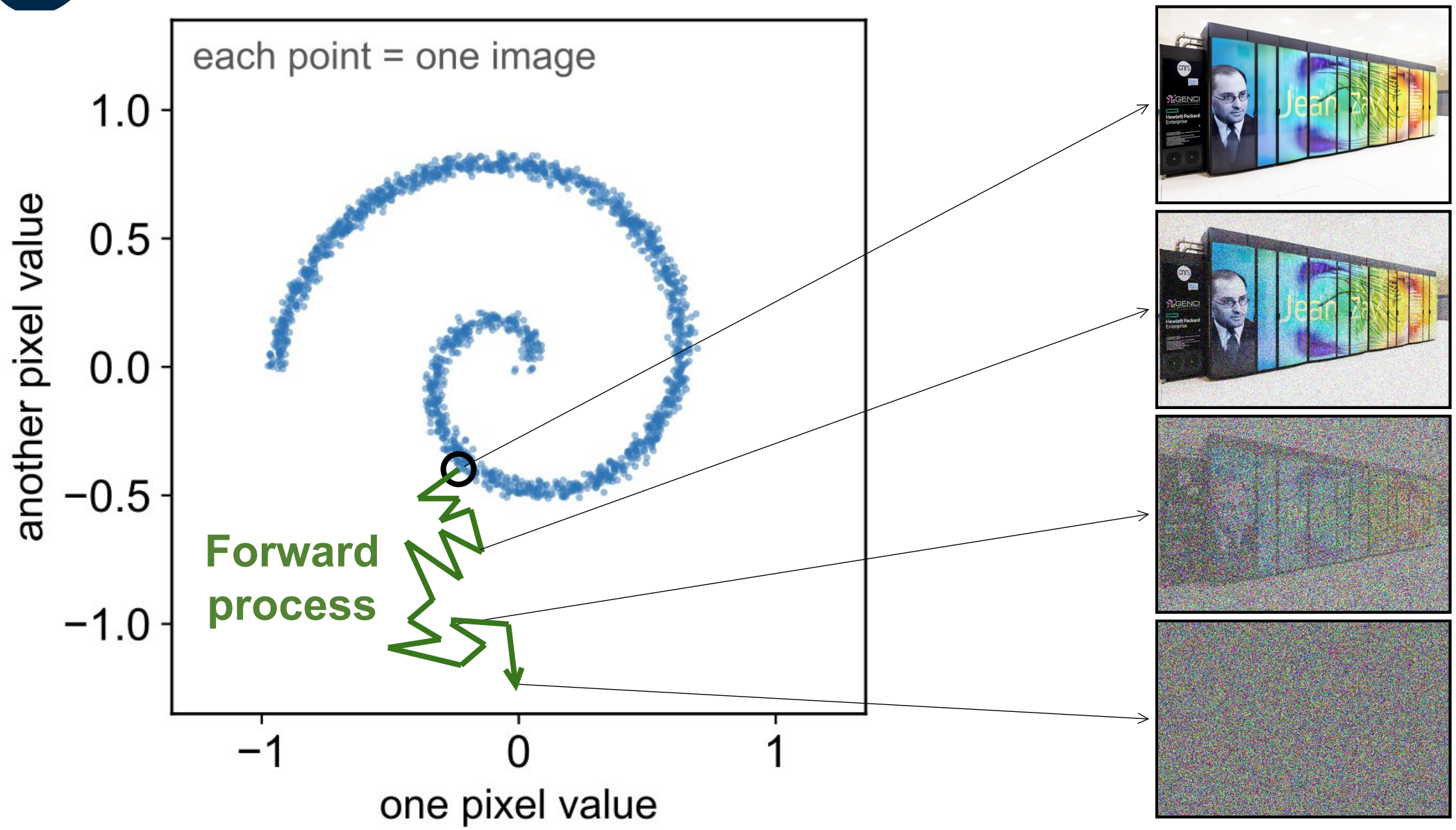
DDPM: Ho et al., 2020 · idea from non-equilibrium thermodynamics: Sohl-Dickstein et al., 2015

Diffusion = two processes

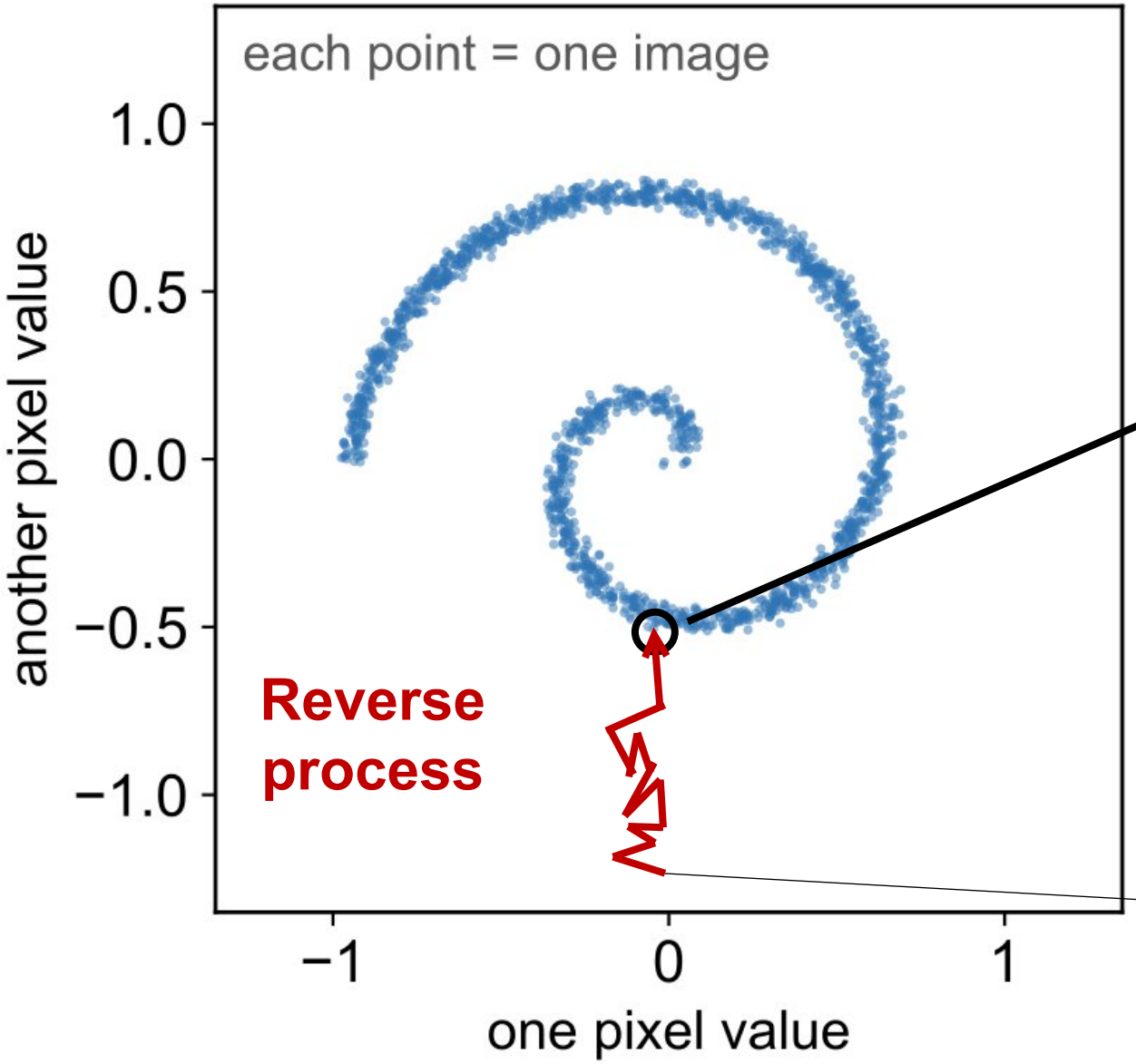




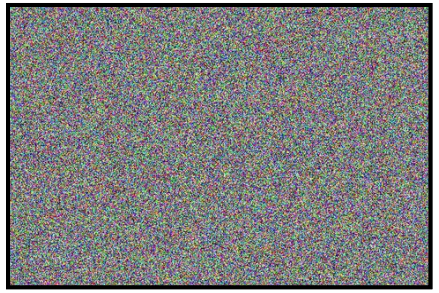
Diffusion = two processes



Diffusion = two processes



Hopefully a nice image of the Jean Zay cluster!



Diffusion = a recipe, not an architecture

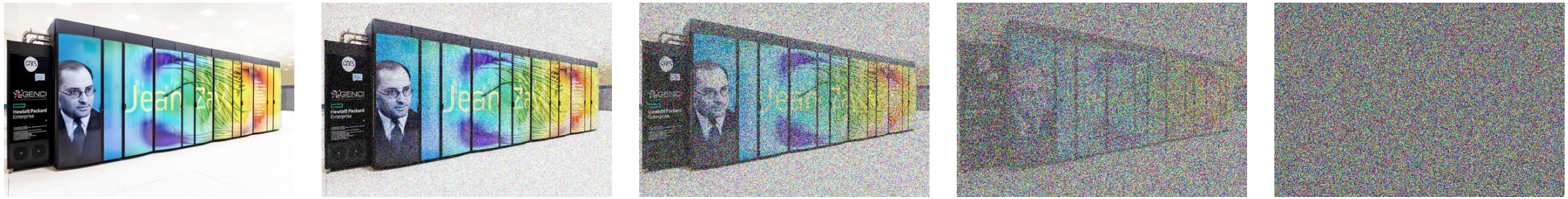
Architectures (what the network is)	Diffusion (how we train and use it)
• Convolutional network (CNN) • Transformer • Graph neural network (GNN) → these are building blocks	**Training:** learn from corrupted data **Sampling:** iteratively transform noise into data Repeat T times → a new sample

Different backbones can implement the denoiser

“Diffusion model” = the framework (corruption + objective + sampler), the network inside is just the denoiser

Forward: destroy the image

$$x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_t \rightarrow \dots \rightarrow x_T$$



t = 0 **t = 100** **t = 250** **t = 500** **t = 1000**

- Add a bit more noise at every step, **T = 1000 is a hyperparameter** (historical DDPM choice)
- t = noise level
- **Fixed process: no parameter to learn**, no network involved
- At the final noise level, the distribution is close to N(0, I)

Vignettes recomputed with the real linear schedule (β_1 = 1e-4 → β_T = 0.02) · photo: Jean Zay, IDRIS/GENCI

Forward: one step

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, z, \qquad z \sim \mathcal{N}(0, I)$$

Signal

$\sqrt{(1-\beta_t)}$ × previous state x_{t-1}

Fresh noise

$\sqrt{\beta_t}$ × new Gaussian noise z

Shrinking the signal keeps the overall scale controlled

Shortcut: jump to any t

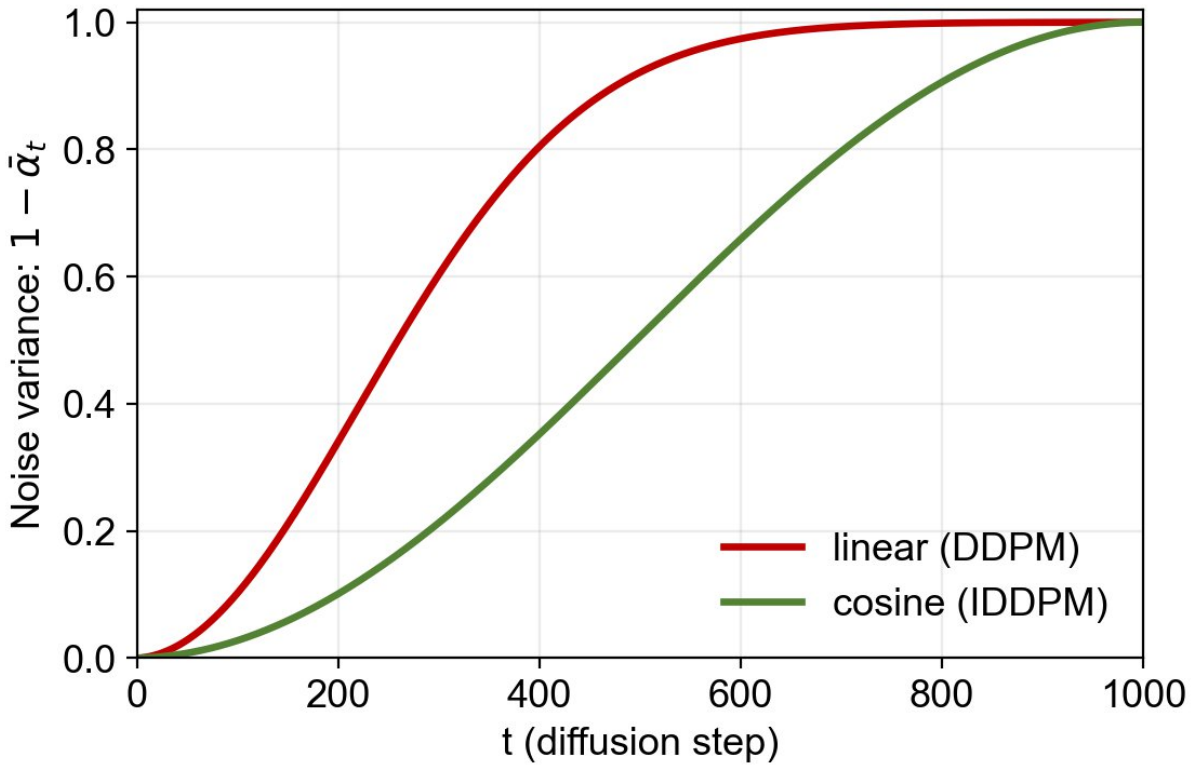
$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

- Independent Gaussian noise terms combine into one Gaussian
- Signal coefficient and noise standard deviation vary with t
- One operation creates a training example at any noise level

★ **Formula 1 of 2: add noise**

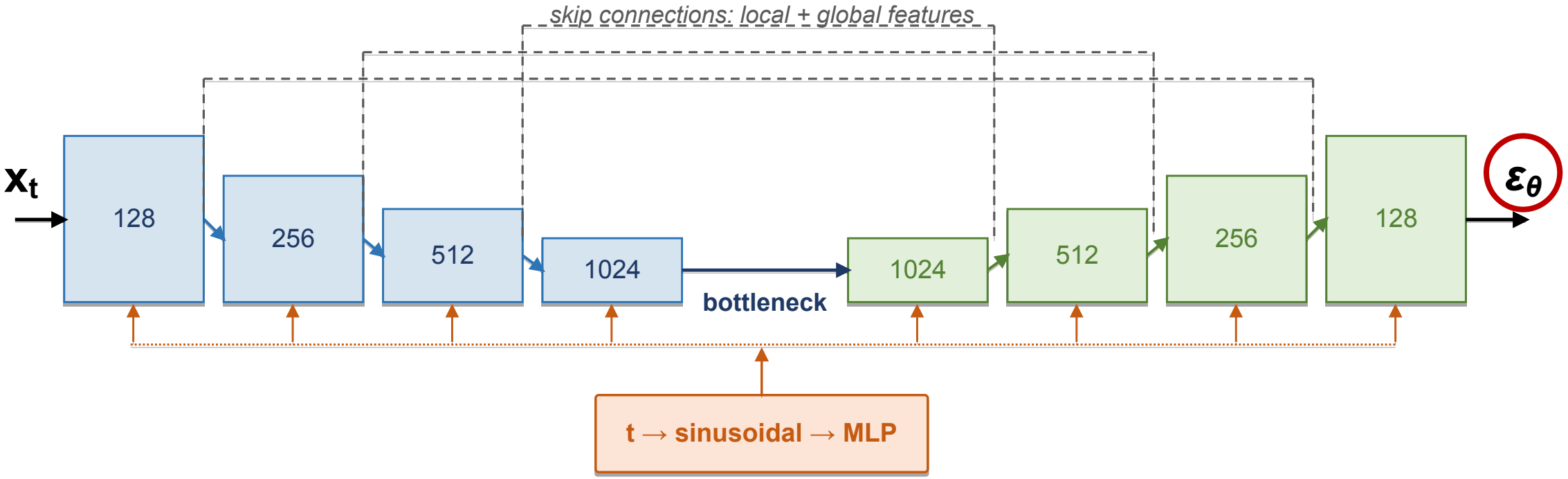
Noise schedule: linear vs cosine



- β_t sets **how fast** the image is destroyed
- DDPM linear: $\beta_1 = 10^{-4} \rightarrow \beta_T = 0.02$
- Here, the linear schedule removes signal quickly
- Cosine preserves useful signal for more of the trajectory

- **The schedule controls the noisy inputs seen at each t**
- **The vertical axis shows the noise variance**

The denoiser: U-Net + time embedding



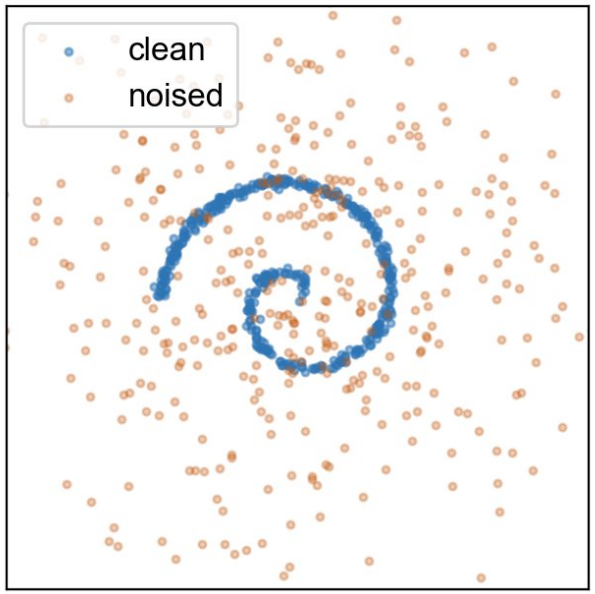
- t → sinusoidal embedding → MLP → injected into residual blocks
- Same resolution in and out ; skip connections combine local and global features
- Numbers: example channel counts ; alternatives include Transformers

Schematic redrawn for this course · U-Net: Ronneberger et al., 2015 · DDPM: Ho et al., 2020

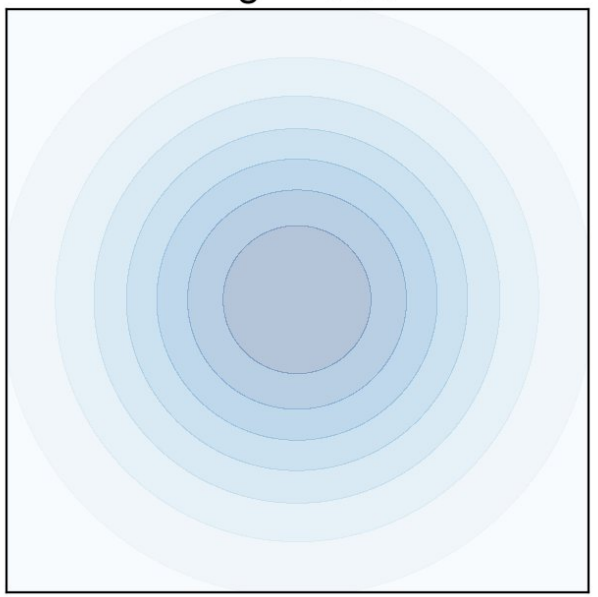
The 2D view: the network learns arrows

Score directions at two noise levels · arrow lengths normalized

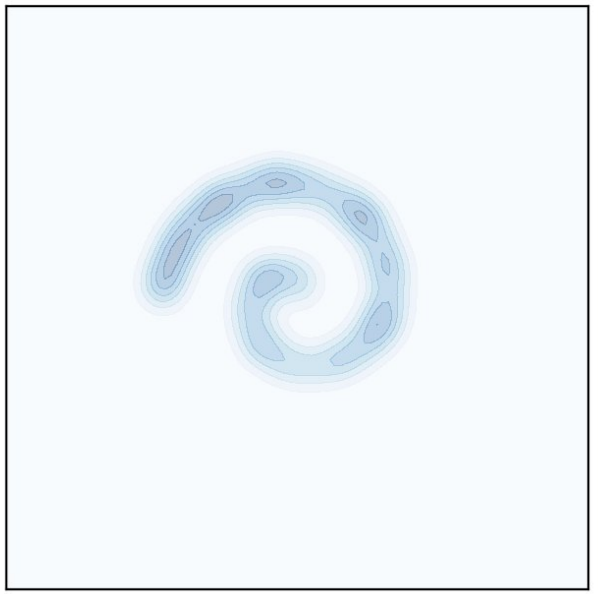
Clean and noised samples



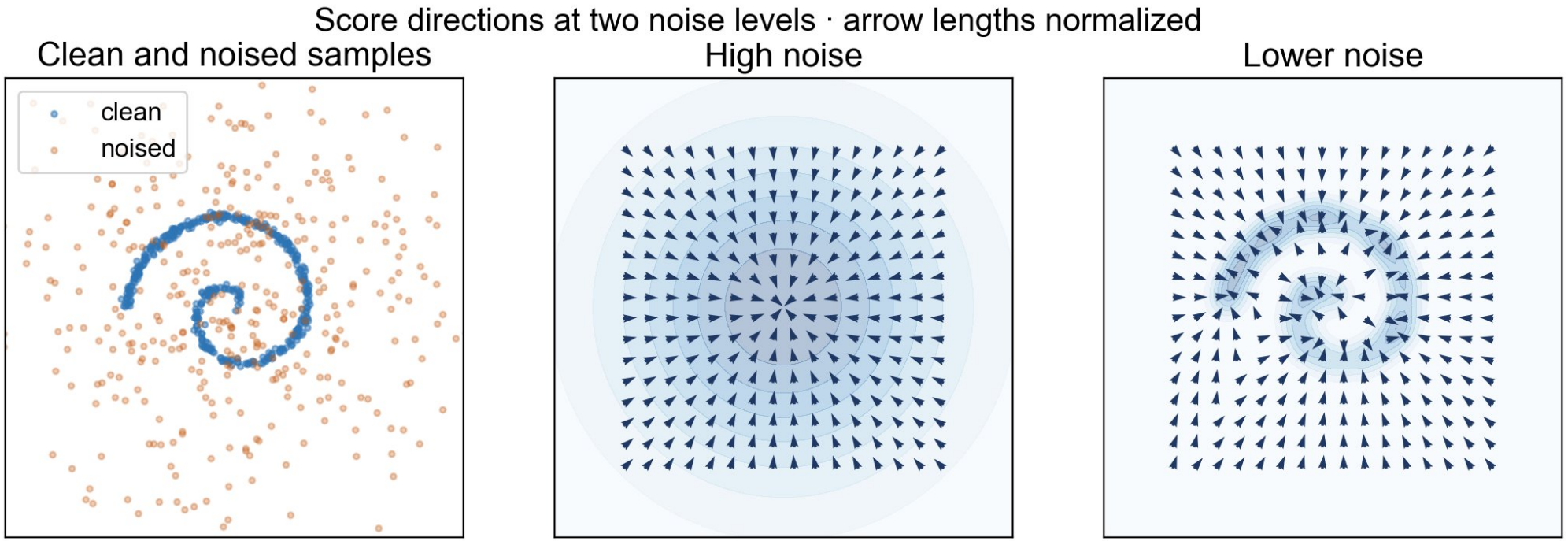
High noise



Lower noise



The 2D view: the network learns arrows



$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t)$$

The score points toward higher density in the noisy distribution

Exact empirical toy score · arrow lengths normalized to show direction

Training: predict the noise you added

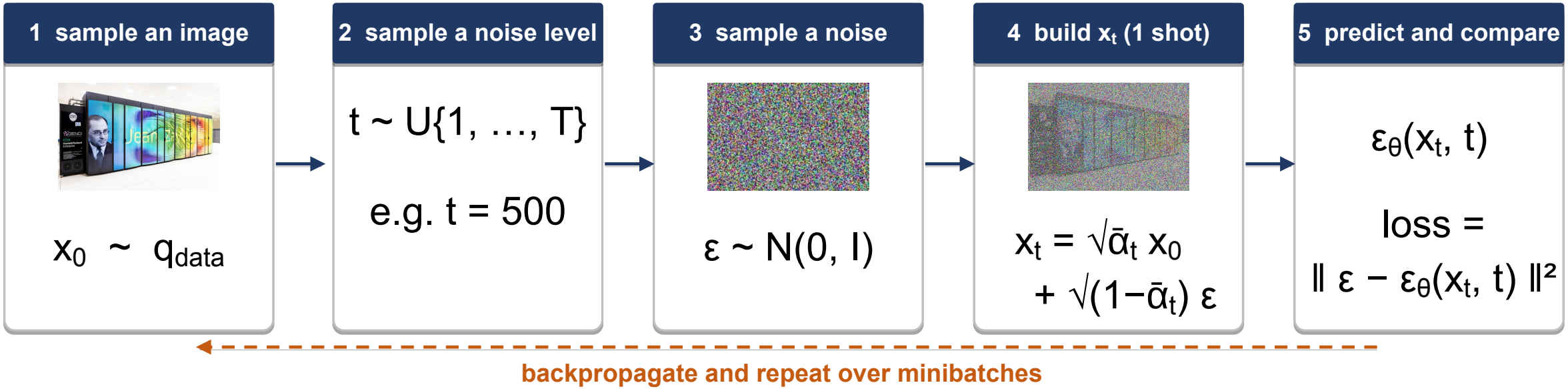
$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\, t,\, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$$

One training step:

- Sample image x_0, noise level t and Gaussian noise ε
- Construct x_t using the shortcut
- Compare network noise prediction ε_θ to real noise ε
- One network evaluation and backpropagation per batch
- No unrolling of the full diffusion trajectory

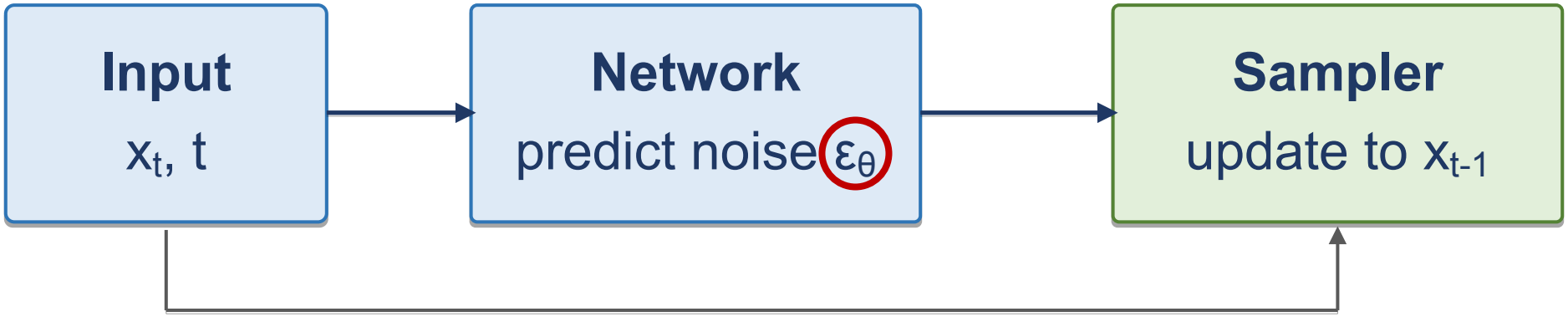
★ **Formula 2 of 2: learn to predict the noise you added**

Training, step by step



- Sample t independently per image ; training covers all noise levels
- Loss = MSE between the true noise and its prediction (pixels, channels, batch)
- Compute depends on model and data ; optionally keep an EMA

One prediction, then one sampling update



- Network: predicts noise
- Sampler: uses x_t, t, prediction and optional fresh noise
- The same network is reused at every noise level

Repeat N times: prediction + sampling update

From predicted noise to one generation step

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$$

- Prediction sets the reverse mean ; DDPM adds fresh noise
- σ_t: fresh-noise standard deviation, specified by the sampler
- $z \sim N(0,I)$
- No fresh noise in the final step

Use mean + noise as the next step and repeat the process

Sampling: the loop

Algorithm 2 · DDPM sampling

1. $x_T \sim N(0, I)$
2. for t = T, …, 1:
3. $\quad z \sim N(0, I)$ if t > 1, else z = 0
4. $\quad x_{t-1} = (x_t - \beta_t\, \varepsilon_\theta(x_t,t)/\sqrt{(1-\bar{\alpha}_t)})/\sqrt{\alpha_t} + \sigma_t\, z$
5. return x_0

Sampler

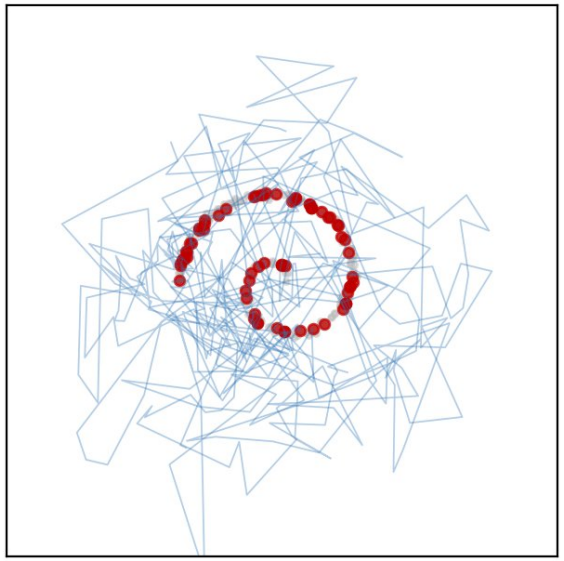
- One network call per step
- Weights fixed at inference
- Fresh noise except final step

Training samples one noise level ; sampling visits decreasing levels

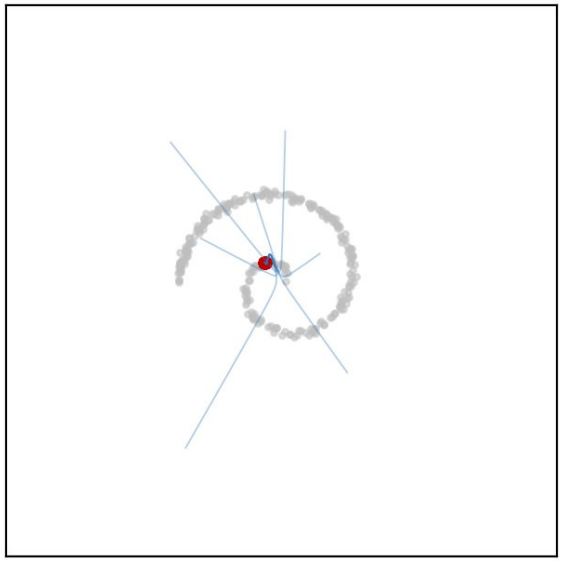
Sampling beyond the predicted mean

Grey: target · blue: selected trajectories · red: final samples

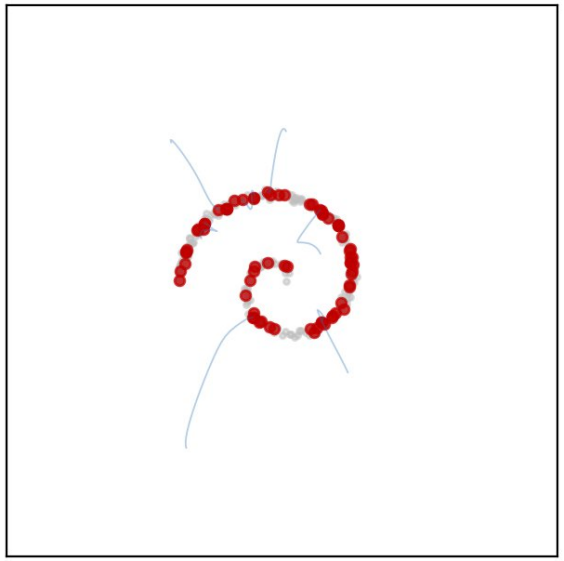
DDPM: stochastic



DDPM: noise removed



DDIM: deterministic, η=0



- DDPM samples around a mean ; removing its noise changes the distribution
- DDIM (Denoising Diffusion Implicit Models) uses a different deterministic update ; initial random noise still gives diversity

Calculated toy example · same starting points · 1,000 full updates

Classifier guidance

$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t)$$

$$s_{\text{guided}}(x_t, t, y) = s_\theta(x_t, t) + \lambda \nabla_{x_t} \log p_\phi(y \mid x_t, t)$$

- **Training:** fit a diffusion model **and** a classifier on noisy inputs
- **Sampling:** differentiate the target-class log-probability with respect to the sample
- Stronger guidance often improves class adherence at the cost of diversity

Dhariwal & Nichol, 2021 · Diffusion Models Beat GANs on Image Synthesis

Classifier-free guidance (CFG)

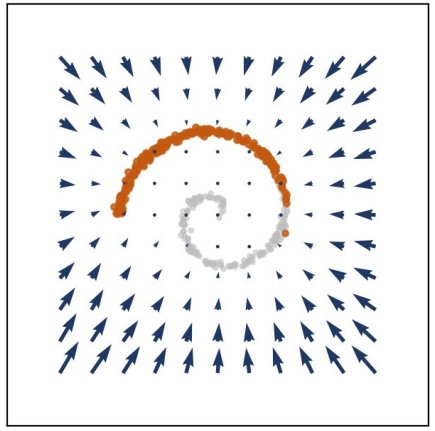
$$\hat{\varepsilon} = \varepsilon_\theta(x_t, t, \varnothing) + w[\varepsilon_\theta(x_t, t, y) - \varepsilon_\theta(x_t, t, \varnothing)]$$

conditional prediction

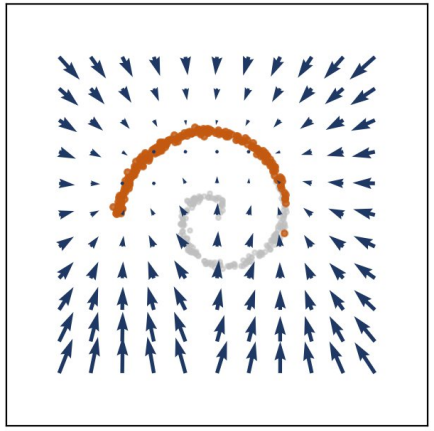
unconditional prediction

Exact toy scores · same arrow scale · not network outputs

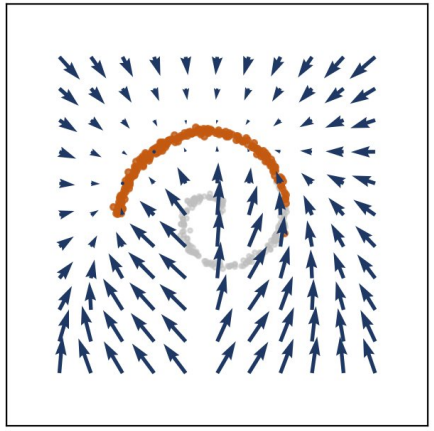
Unconditional score



Conditional: radius ≥ 0.66



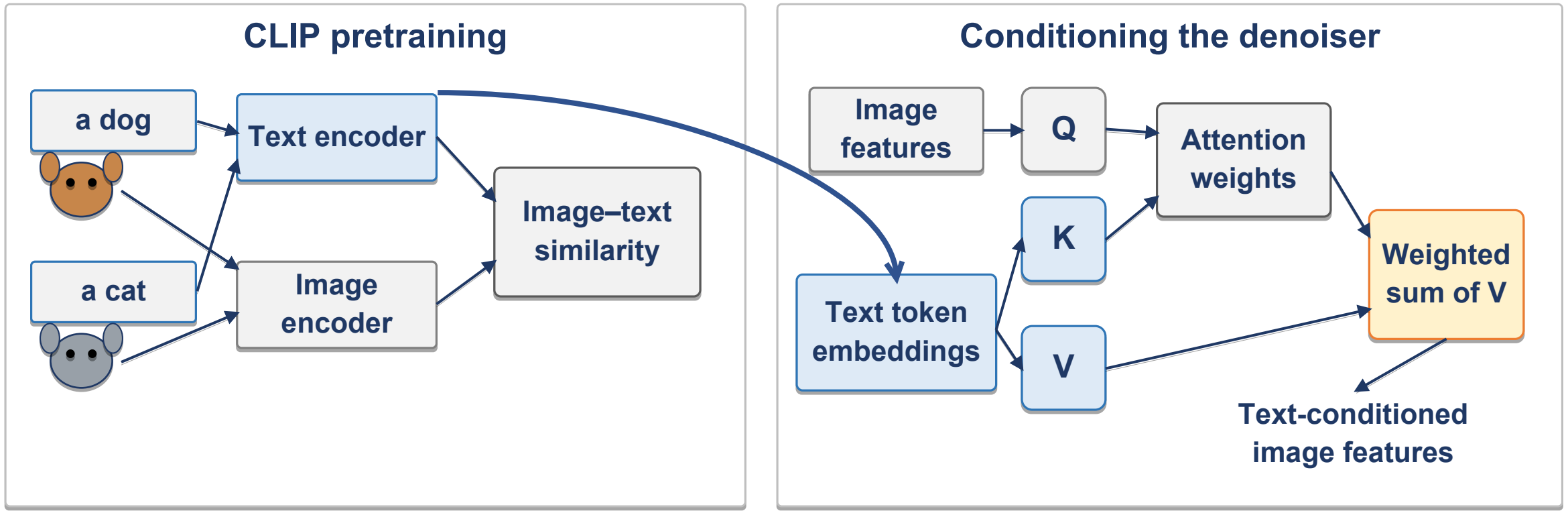
Guided score, w = 3



- w = 0: unconditional
- w = 1: conditional
- w > 1: amplified conditioning
- One network, trained with y or ∅

Two predictions per step ; useful strength depends on the model

Text conditioning: CLIP + cross-attention

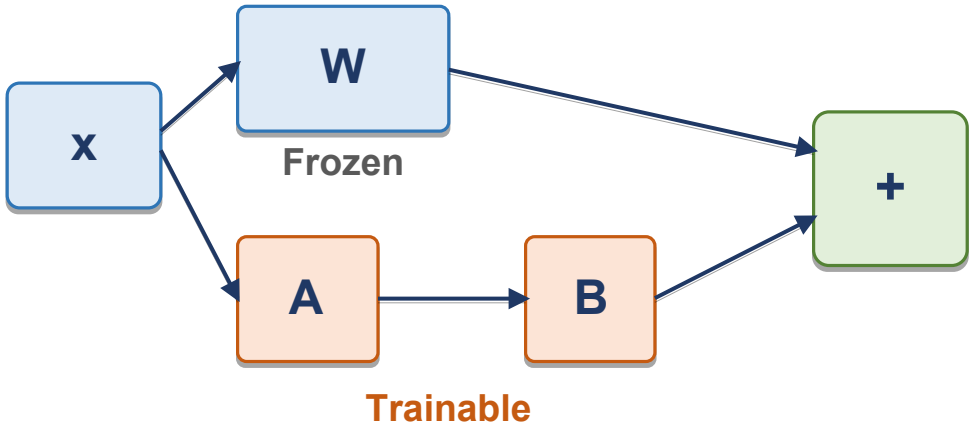


Q and K determine the weights; these weights combine V

CLIP: Radford et al., 2021 · drawings are schematic

Adapting a pretrained model

LoRA · low-rank update



$A: r \times d_{in} \cdot B: d_{out} \times r$

ControlNet · spatial condition



Zero-initialized projections add features

Fine-tuning: monitor generations and overfitting, not only loss

The sampling bottleneck

Family	Typical strengths	Common challenges
GAN	Fast sampling, high fidelity	Training stability, mode coverage
VAE	Fast sampling, useful latent space	Reconstruction / detail trade-offs
Diffusion	Sample quality and diversity	Repeated network evaluations

Historical overview, not a universal ranking

Two levers: cheaper evaluations & fewer evaluations

Faster: latent diffusion + DDIM

Latent diffusion

Training



512×512×3

Encoder

Clean latent
64×64×4

Noise + train denoiser

Generation

latent noise

Denoising loop

Decoder



image (illustrative)

DDIM · timestep grid

Many small steps



Selected noise levels



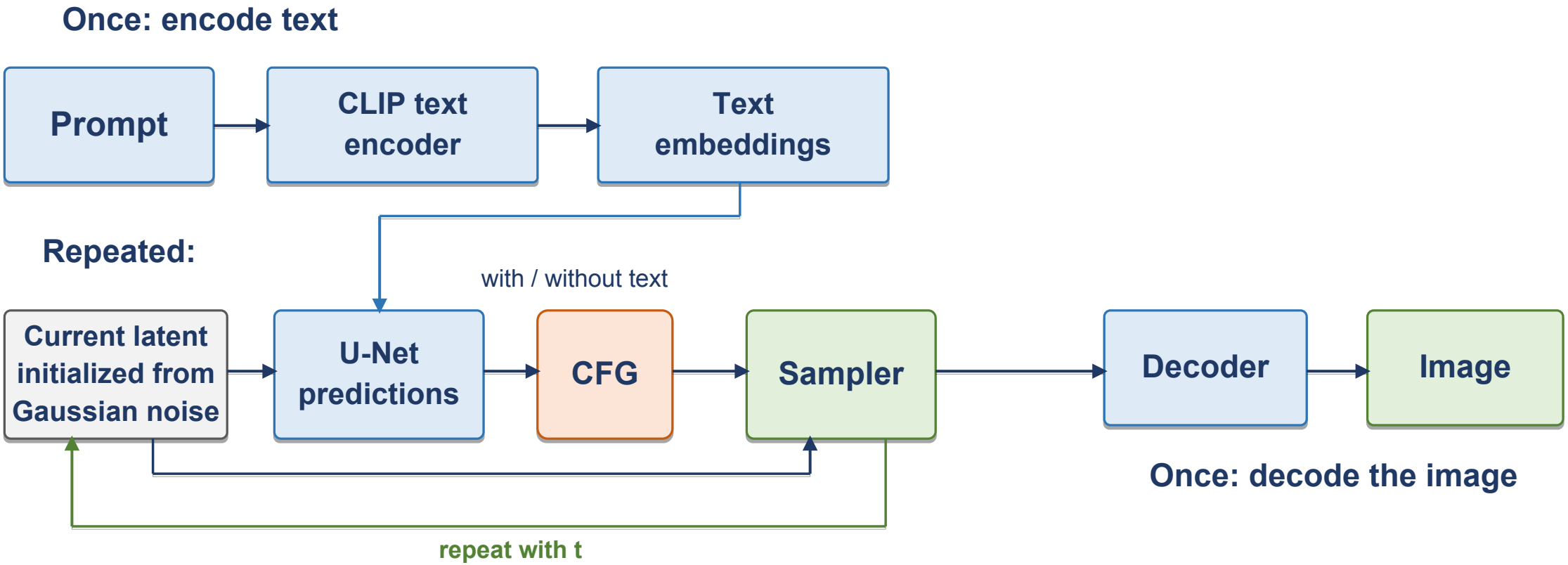
Same denoiser, different update and timestep grid

Beyond DDPM: backbones and objectives

Backbone	Objective	Data
U-Net or Transformer	Noise prediction or **flow matching**	Images, video, structures, weather

- Flow matching learns a velocity field along a chosen interpolation
- Training interpolations may be straight; generated trajectories need not be
- Learn to transform a simple distribution into data

Putting the pieces together: Stable Diffusion v1

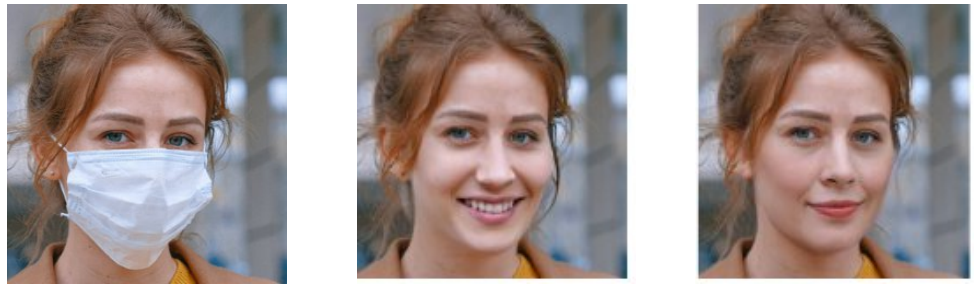


U-Net ≈ 860M · CLIP text encoder ≈ 123M · ~150,000 A100 GPU-hours

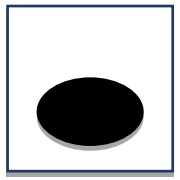
CompVis model card · SD v1-4 · LAION-2B-en and filtered subsets

Editing with masks: RePaint

One observation · two completions

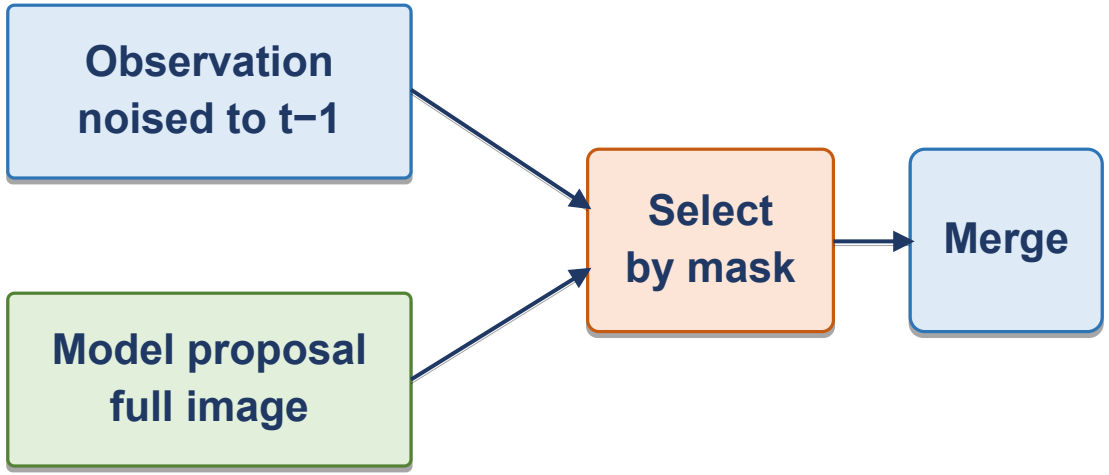


Observation | Completion 1 | Completion 2



White: observed
Black: edited (schematic)

One sampling update



Both inputs at noise level t−1

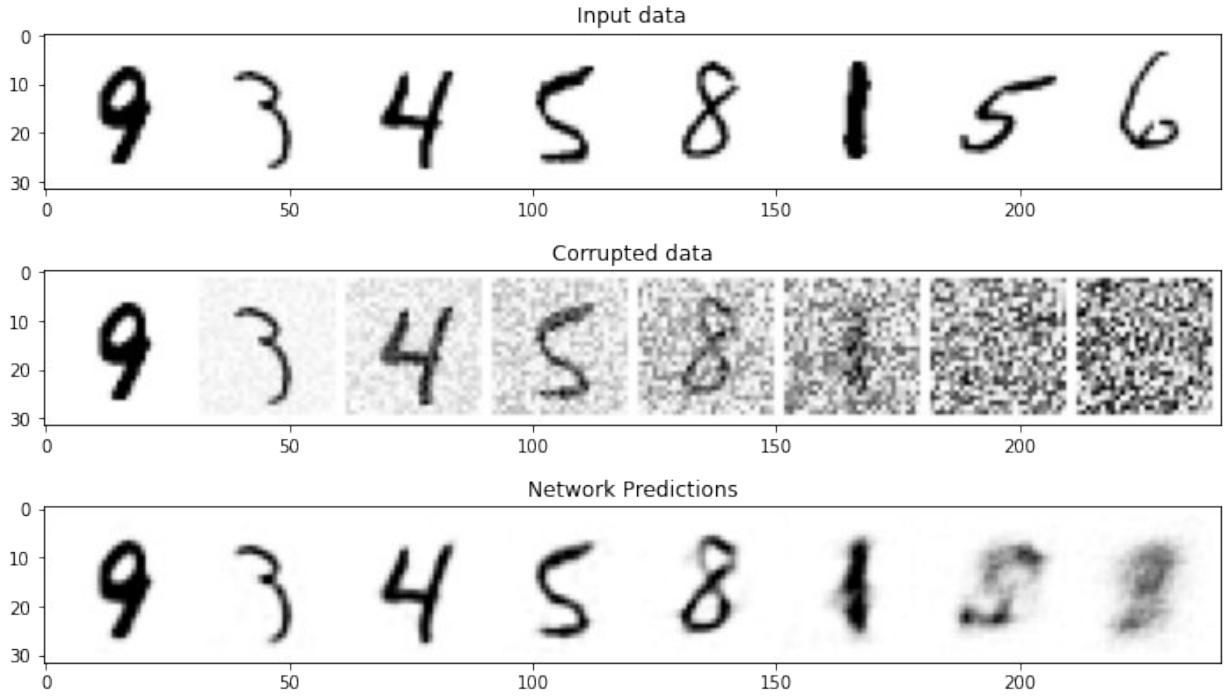
Same pretrained model · observed pixels enforced during sampling

Lugmayr et al., RePaint, CVPR 2022 · https://arxiv.org/abs/2201.09865

Take-aways & resources

1. Generation aims to reproduce a distribution and its diversity
2. Training: corrupt data and learn to predict added noise
3. Sampling: combine predictions with a compatible update rule
4. Random initial noise allows diversity with deterministic sampling
5. Conditioning steers; latent spaces and fast samplers reduce cost

Annex — One-shot MSE predicts an average



Strong noise: several clean images are plausible

An MSE predictor averages these possibilities

Take a small step, then predict again

Hugging Face Diffusion Course, Unit 1 (MNIST)

What should the network predict?

Noise ε	Clean image x_0	v-prediction
DDPM · SD 1.x	Denoised-image parameterization	Linear signal / noise combination Example: SD 2 at 768 px

For fixed noising, predictions can be converted into each other

Unweighted MSE losses emphasize noise levels differently

Flow-matching velocity follows a different path — slide 31

Predicting the noise and learning the score

$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t)$$

$$\mu_\theta(x_t, t) = \frac{x_t + \beta_t s_\theta(x_t, t)}{\sqrt{\alpha_t}}$$

The score describes the noisy distribution at time t

Noise prediction and score estimation differ by a known scaling

Guidance changes the sampling field, not weights during inference

In practice with diffusers


```
import torch
from diffusers import DiffusionPipeline, DPMSolverMultistepScheduler

pipe = DiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16, variant="fp16",
).to("cuda")
pipe.scheduler = DPMSolverMultistepScheduler.from_config(
    pipe.scheduler.config, algorithm_type="dpmsolver++", solver_order=2,
)
image = pipe(
    "a photo of colorful supercomputer server racks",
    num_inference_steps=25, guidance_scale=6.0,
    generator=torch.Generator(device="cuda").manual_seed(0),
).images[0]
```

Checkpoint + compatible sampler + prompt + random seed

Annex — the reverse posterior, in detail

Bayes + Markov on $q(x_{t-1} \mid x_t, x_0)$ — detail of slide 16

$$q(x_{t-1} \mid x_t, x_0) = q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0) \,/\, q(x_t \mid x_0)$$

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(x_{t-1} ; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t\, I \right)$$

$$\tilde{\beta}_t = (1 - \bar{\alpha}_{t-1}) \,/\, (1 - \bar{\alpha}_t) \cdot \beta_t$$

$$\tilde{\mu}_t = \left(\sqrt{\bar{\alpha}_{t-1}}\, \beta_t \,/\, (1 - \bar{\alpha}_t) \right) x_0 + \left(\sqrt{\alpha_t}\, (1 - \bar{\alpha}_{t-1}) \,/\, (1 - \bar{\alpha}_t) \right) x_t$$

$$\text{with } x_0 = \left(x_t - \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon \right) / \sqrt{\bar{\alpha}_t} : \quad \tilde{\mu}_t = (1/\sqrt{\alpha_t}) \left(x_t - (\beta_t / \sqrt{(1-\bar{\alpha}_t)})\, \varepsilon \right)$$

Exact given x0 · do not confuse with q(xt-1 | xt)

Annex — variational bound & simplified loss

$$\mathcal{L}_{\text{VLB}} = \mathbb{E}_q\left[\log\left(\frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{0:T})}\right)\right] \geq -\mathbb{E}_{q(x_0)}[\log p_\theta(x_0)]$$

$$L_{t-1} = \mathbb{E}\left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right] + C$$

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon}[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$$

Fixed σ_t^2, $t > 1$: denoising terms are weighted MSEs

Simple loss changes the weights and optimizes a different objective

Ho et al., 2020, sections 3.2–3.4 · https://arxiv.org/abs/2006.11239

Annex — guidance with a differentiable loss

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$\varepsilon_{\text{guided}} = \varepsilon_\theta + \lambda_t \sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t}\ell(\widehat{x}_0)$$

Predict clean image → evaluate constraint → differentiate

Adjust noise prediction → one scheduler step

Weights frozen · strength must be tuned

Pixel-space ε model and compatible DDIM scheduler; see supplied code

Illustrative example: scripts/example_guidance.py

Annex — classifier guidance in 4 steps

1. Train the unconditional diffusion model
2. Train a classifier on noisy images x_t (it must know t)
3. Add $\lambda \cdot \nabla \log p_\varphi(y \mid x_t, t)$ to the unconditional score
4. Denoise with the guided score

Limits: a second model to train, a backward pass per step, a fixed label set, sensitivity to adversarial directions

Annex — Textual Inversion & DreamBooth

Textual Inversion

Learn one or a few token embeddings from reference images

Keep the generative model frozen

Small file tied to the text encoder and base model

DreamBooth

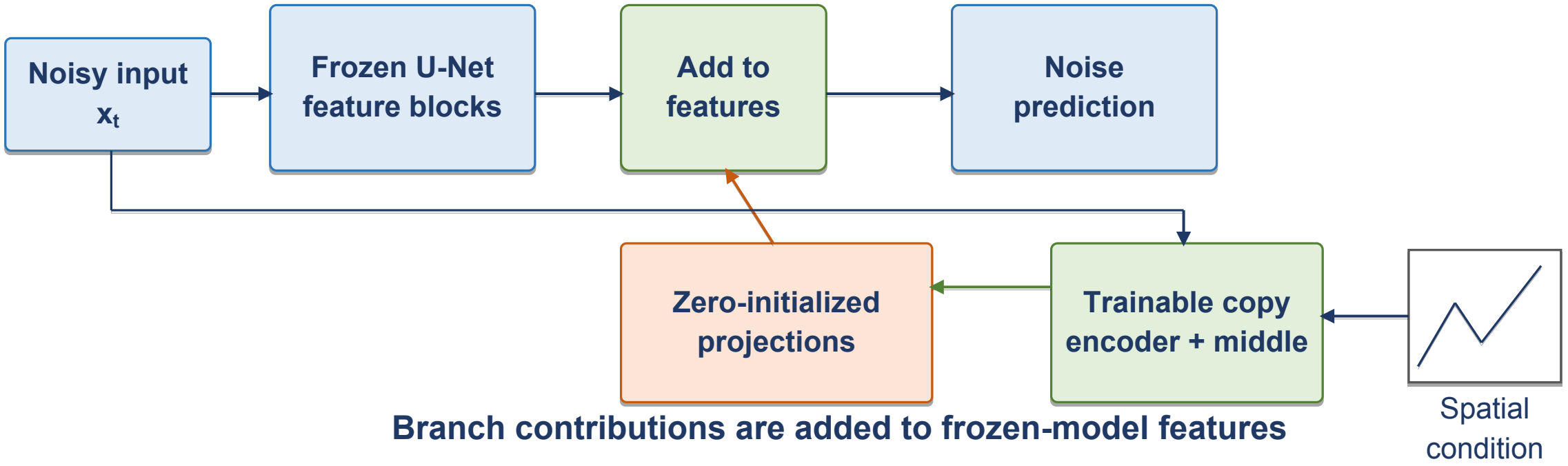
Personalize a model to a specific subject

Original method: fine-tuning with prior preservation

Often implemented with LoRA

Gal et al., 2022 · Ruiz et al., 2022

Annex — ControlNet in detail



Zero-initialized projections preserve the base output at initialization

Simplified schematic adapted from Zhang et al., 2023 · arxiv.org/abs/2302.05543

Annex — DDIM sampling rule

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta}{\sqrt{\bar{\alpha}_t}}$$

$$\sigma_{t\to s} = \eta \sqrt{\frac{1-\bar{\alpha}_s}{1-\bar{\alpha}_t}} \sqrt{1-\frac{\bar{\alpha}_t}{\bar{\alpha}_s}}$$

$$x_s = \sqrt{\bar{\alpha}_s}\,\widehat{x}_0 + \sqrt{1-\bar{\alpha}_s-\sigma_{t\to s}^2}\,\varepsilon_\theta + \sigma_{t\to s} z$$

$s < t$ · consecutive steps: $\sigma_t^2 = \eta^2\,\tilde{\beta}_t$

$\eta = 0$: deterministic · same denoiser, different update

Use a coarser time grid; fewer steps trade accuracy for speed

Song et al., 2021 · https://arxiv.org/abs/2010.02502

Annex — masks for editing tasks

Inpainting



Black: to generate

Outpainting



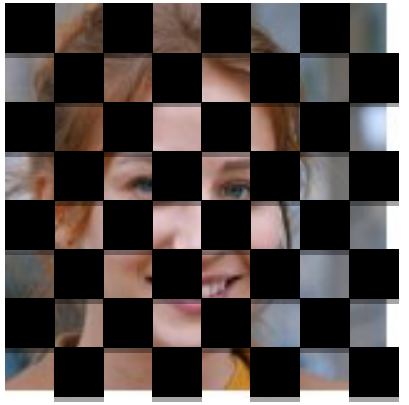
Black: to generate

Missing rows



Black: to generate

Sparse pixels



Black: to generate

Sparse-pixel reconstruction is one special case of super-resolution

Schematic masks on one source image · Lugmayr et al., RePaint, CVPR 2022

Annex — RePaint: one step in detail

$$x_{t-1} = m \odot x_{t-1}^{\text{known}} + (1 - m) \odot x_{t-1}^{\text{model}}$$

m = 1: observed region · both parts are at noise level t−1

Re-noise observations, sample unknown region, merge and continue

Resampling alternates forward noising and reverse updates

Lugmayr et al., RePaint, CVPR 2022

Annex — extended resources

Foundations · DDPM

Score-based SDEs

Samplers · DDIM

DPM-Solver++

Guidance · CFG

ControlNet

Latent diffusion

Diffusion Transformer (DiT)

Flow Matching

Rectified Flow

Intuition · Welch Labs / 3Blue1Brown

Practice · Hugging Face diffusion course

Click a resource to explore the topic